

Modele ML

May 18, 2026

1 1. Importation des bibliothèques

```
[270]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

from sklearn.ensemble import RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.neural_network import MLPRegressor
from xgboost import XGBRegressor

from IPython.display import HTML
import os
```

2 2. Chargement et Exploration de dataset df_complet_clean.csv

```
[271]: folder_path = "VF_Dataset"
df_complet = pd.read_csv(f"{folder_path}/df_complet_clean.csv")
```

```
[272]: print("Dimensions de dataset complet :", df_complet.shape)
print("Nombre de Communes :", df_complet["libelle_commune"].nunique())
```

Dimensions de dataset complet : (1845, 87)
Nombre de Communes : 123

```
[273]: HTML("""
<div style="overflow:auto; width:100%; height:300px; border:1px solid black">
{}
</div>
""").format(df_complet.to_html())
```

[273]: <IPython.core.display.HTML object>

3 3. Traitement des valeurs NULL —> NULL to Median

```
[274]: cols_num = df_complet.select_dtypes(
        include=["int64", "float64"]
    ).columns

for col in cols_num:
    median_value = df_complet[col].median()
    df_complet[col].fillna(median_value, inplace=True)
```

C:\Users\ZAKARIA\AppData\Local\Temp\ipykernel_28804\74089255.py:7:
FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df_complet[col].fillna(median_value, inplace=True)
```

4 4. Suppression des abstentions, blancs et nuls

```
[275]: pd.DataFrame(df_complet["nom_complet"].unique(), columns=["nom_complet"])
```

```
[275]:      nom_complet
0    ARTHAUD Nathalie
1      abstentions
2  MÉLENCHON Jean-Luc
3        blancs
4    ROUSSEL Fabien
5      HIDALGO Anne
6    LASSALLE Jean
```

```

7    DUPONT-AIGNAN Nicolas
8          LE PEN Marine
9    PÉCRESSE Valérie
10          ZEMMOUR Éric
11          JADOT Yannick
12    POUTOU Philippe
13    MACRON Emmanuel
14          nuls

```

```

[276]: to_remove = ["abstentions", "blancs", "nuls"]

df_complet = df_complet[~df_complet["nom_complet"].isin(to_remove)]

print(df_complet.shape)

```

(1476, 87)

```

[277]: pd.DataFrame(df_complet["nom_complet"].unique(), columns=["nom_complet"])

```

```

[277]:      nom_complet
0    ARTHAUD Nathalie
1    MÉLENCHON Jean-Luc
2    ROUSSEL Fabien
3    HIDALGO Anne
4    LASSALLE Jean
5    DUPONT-AIGNAN Nicolas
6    LE PEN Marine
7    PÉCRESSE Valérie
8    ZEMMOUR Éric
9    JADOT Yannick
10   POUTOU Philippe
11   MACRON Emmanuel

```

5 5. Extraction de dataset _Gagnant + Exploration

```

[278]: df_gagnant = df_complet.loc[
        df_complet.groupby("libelle_commune")["voix"].idxmax()
    ]

```

```

[279]: folder_path = "VF_Dataset"
os.makedirs(folder_path, exist_ok=True)
df_gagnant.to_csv(f"{folder_path}/df_gagnant.csv", index=False)

```

```

[280]: print("Dimensions de dataset gagnant :", df_gagnant.shape)
print("Nombre de Communes :", df_gagnant["libelle_commune"].nunique())

```

Dimensions de dataset gagnant : (123, 87)
 Nombre de Communes : 123

```
[281]: HTML("""
<div style="overflow:auto; width:100%; height:300px; border:1px solid black">
{}
</div>
""").format(df_gagnant.to_html())
```

```
[281]: <IPython.core.display.HTML object>
```

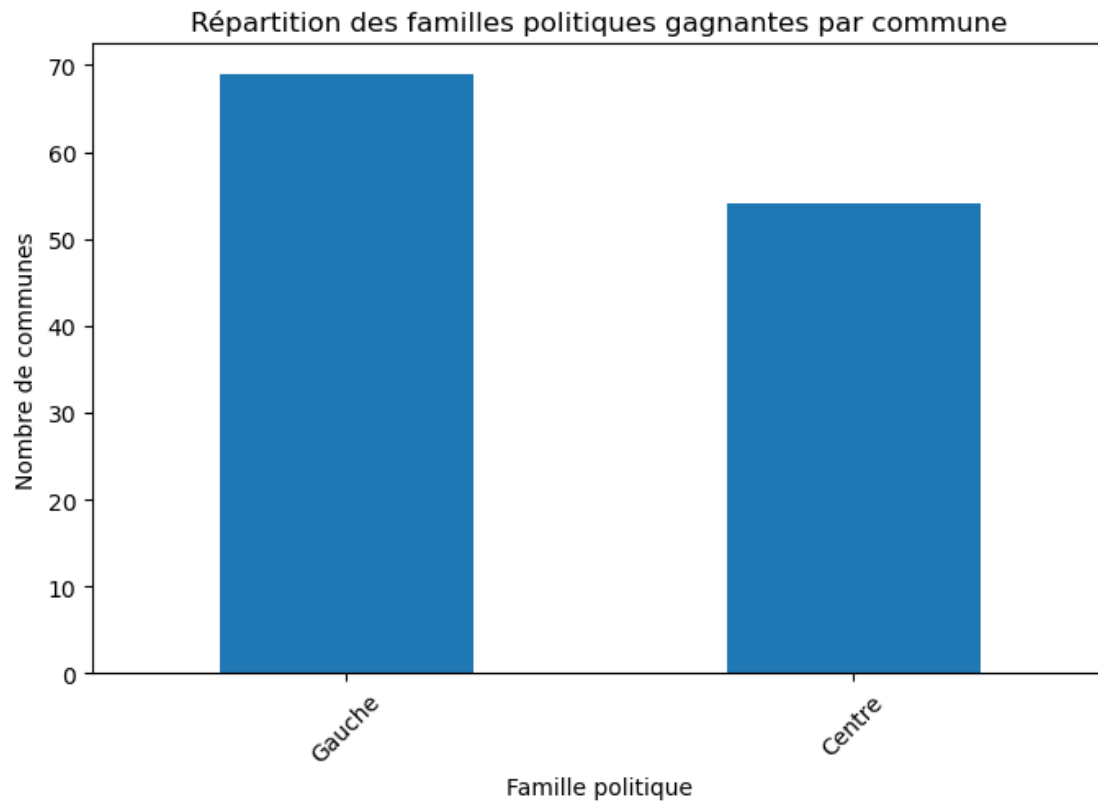
6 6. Répartition des familles politiques

```
[282]: print(df_gagnant["famille_politique"].value_counts())
```

```
famille_politique
Gauche      69
Centre      54
Name: count, dtype: int64
```

```
[283]: df_gagnant["famille_politique"].value_counts().plot(
    kind="bar",
    figsize=(8,5),
    title="Répartition des familles politiques gagnantes par commune"
)

plt.xlabel("Famille politique")
plt.ylabel("Nombre de communes")
plt.xticks(rotation=45)
plt.show()
```



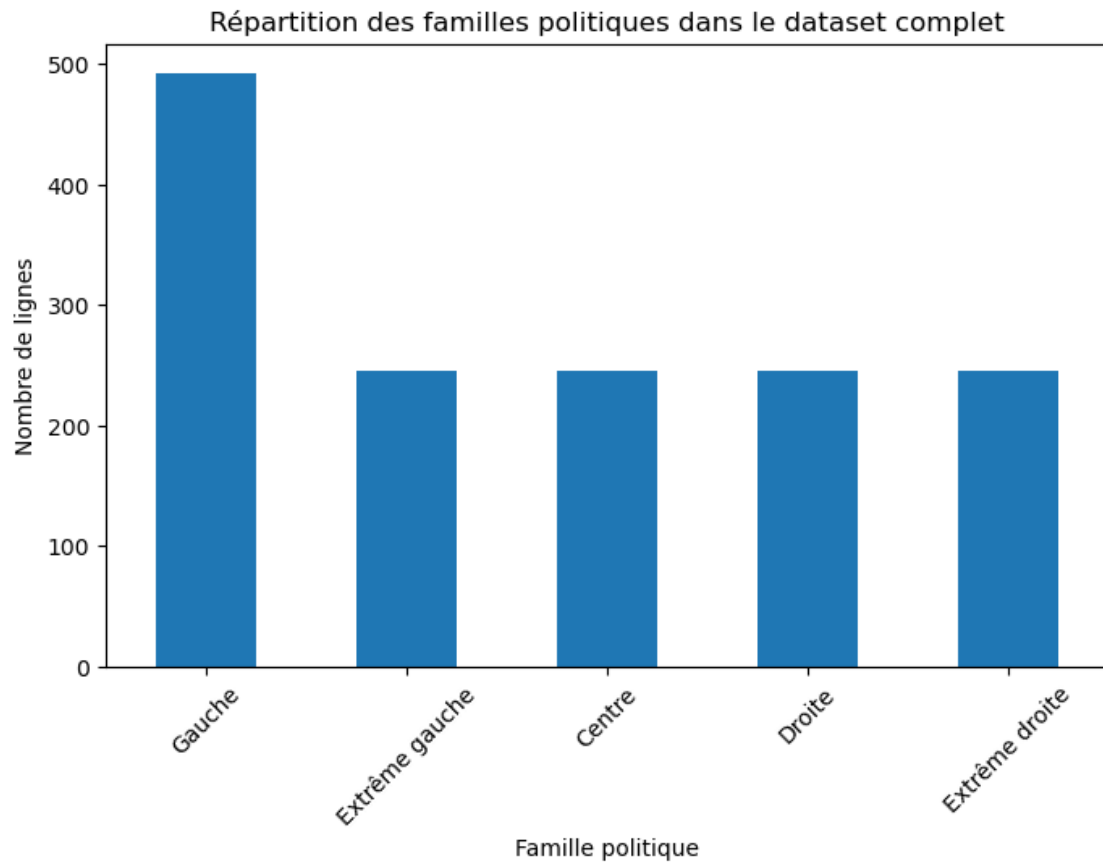
```
[284]: print(df_complet["famille_politique"].value_counts(dropna=False))
```

```
famille_politique
Gauche          492
Extrême gauche  246
Centre          246
Droite          246
Extrême droite  246
Name: count, dtype: int64
```

```
[285]: df_complet["famille_politique"].value_counts().plot(
        kind="bar",
        figsize=(8,5),
        title="Répartition des familles politiques dans le dataset complet"
    )

plt.xlabel("Famille politique")
plt.ylabel("Nombre de lignes")
plt.xticks(rotation=45)

plt.show()
```



7 7. Distribution de la variable cible (voix) et (log_voix)

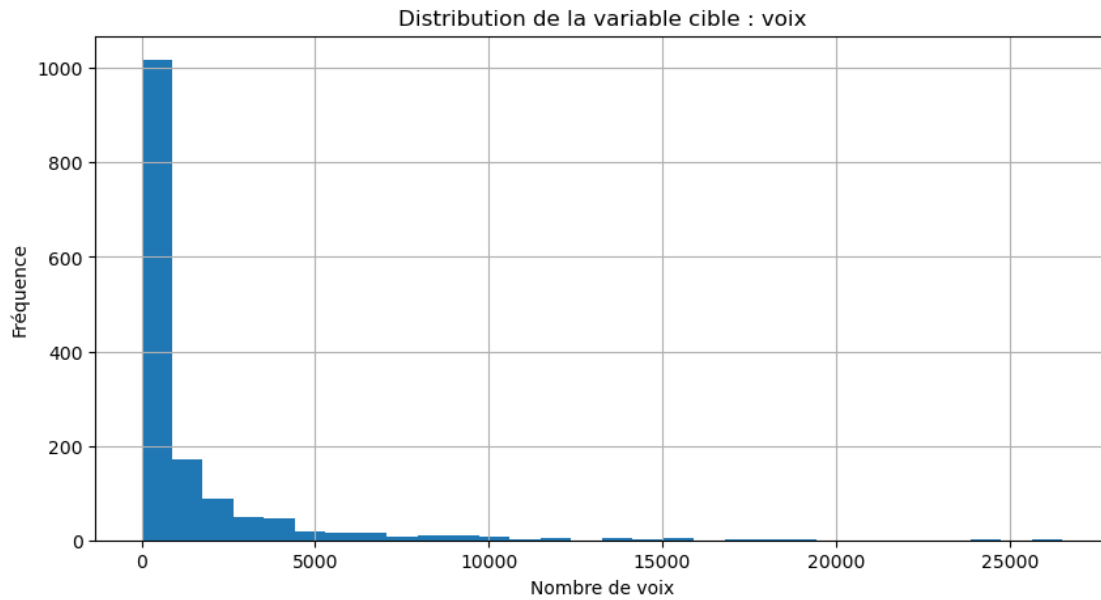
```
[286]: plt.figure(figsize=(10,5))

df_complet["voix"].hist(
    bins=30
)

plt.title("Distribution de la variable cible : voix")

plt.xlabel("Nombre de voix")
plt.ylabel("Fréquence")

plt.show()
```



énorme asymétrie, beaucoup de petites valeurs, quelques très grosses valeurs, outliers très dominants.
- problème classique en régression

7.0.1 Transformation logarithmique de la cible

```
[287]: df_complet["log_voix"] = np.log1p(df_complet["voix"])
```

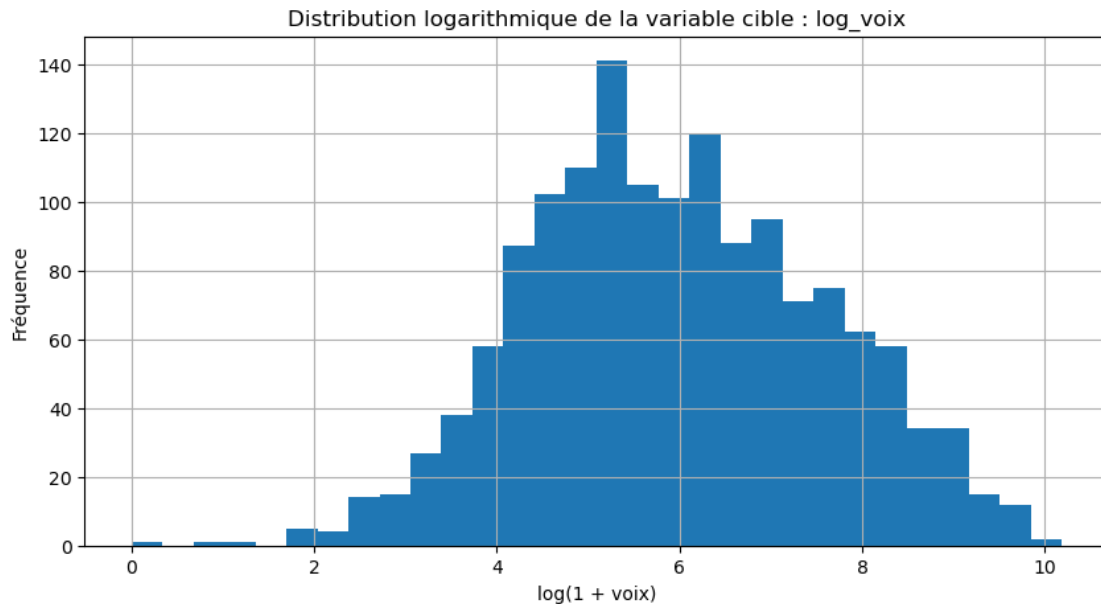
```
[288]: plt.figure(figsize=(10,5))

df_complet["log_voix"].hist(
    bins=30
)

plt.title(
    "Distribution logarithmique de la variable cible : log_voix"
)

plt.xlabel("log(1 + voix)")
plt.ylabel("Fréquence")

plt.show()
```



distribution plus équilibrée variance plus stable réduction impact outliers meilleure séparation des valeurs apprentissage ML plus stable

L'analyse initiale de la variable cible voix a montré une forte asymétrie positive ainsi que la présence de valeurs extrêmes importantes. Afin de réduire l'impact des outliers et de stabiliser la variance des données, une transformation logarithmique $\log(1 + \text{voix})$ a été appliquée. La nouvelle distribution obtenue est beaucoup plus équilibrée et mieux adaptée à l'apprentissage des modèles de Machine Learning. ¶

8 8. Création de nouvelles variables intelligentes

```
[289]: df_complet["entreprises_par_habitant"] = (
        df_complet["nombre_total_etablissements_2023"] /
        df_complet["population_totale_2022"]
    )
df_complet["entreprises_par_habitant"].describe()
```

```
[289]: count      1476.000000
mean         0.088906
std          0.033325
min          0.012636
25%          0.068156
50%          0.080963
75%          0.097795
max          0.248345
Name: entreprises_par_habitant, dtype: float64
```

Le ratio entreprises par habitant permet d'évaluer le niveau d'activité économique des communes.

Les territoires présentant une forte densité d'établissements peuvent présenter des comportements électoraux différents liés à leur attractivité économique et à leur dynamisme local.

```
[290]: df_complet["associations_par_habitant"] = (  
        df_complet["nb_associations"] /  
        df_complet["population_totale_2022"]  
    )  
df_complet["associations_par_habitant"].describe()
```

```
[290]: count      1476.000000  
mean         0.019731  
std          0.006287  
min          0.002808  
25%          0.016174  
50%          0.018143  
75%          0.022474  
max          0.047980  
Name: associations_par_habitant, dtype: float64
```

Le nombre d'associations rapporté à la population permet d'estimer la dynamique sociale et associative des communes.

Cette variable peut refléter un niveau plus élevé de participation citoyenne et d'engagement local.

```
[291]: df_complet["participation_reelle"] = (  
        df_complet["total_votants"] /  
        df_complet["total_inscrits"]  
    )  
df_complet["participation_reelle"].describe()
```

```
[291]: count      1476.000000  
mean         0.749669  
std          0.054304  
min          0.582404  
25%          0.706978  
50%          0.761336  
75%          0.795790  
max          0.856345  
Name: participation_reelle, dtype: float64
```

Le taux de participation réelle mesure le niveau de mobilisation électorale dans les communes.

Cette variable constitue un indicateur important des comportements électoraux et de l'implication citoyenne.

```
[292]: df_complet["taux_logements_vacants"] = (  
        df_complet["nombre_logements_vacants"] /  
        df_complet["nombre_total_logements"]  
    )  
df_complet["taux_logements_vacants"].describe()
```

```
[292]: count      1476.000000
      mean        0.060141
      std         0.018460
      min         0.004145
      25%         0.047202
      50%         0.060220
      75%         0.071102
      max         0.114926
      Name: taux_logements_vacants, dtype: float64
```

Le taux de logements vacants permet d'évaluer certaines fragilités territoriales ainsi que la dynamique résidentielle des communes.

Cette variable peut indirectement refléter l'attractivité économique et sociale des territoires étudiés.

```
[293]: df_complet["taux_hlm"] = (
      df_complet["residences_principales_hlm"] /
      df_complet["nombre_total_logements"]
    )
df_complet["taux_hlm"].describe()
```

```
[293]: count      1476.000000
      mean        0.239711
      std         0.121811
      min         0.036101
      25%         0.151441
      50%         0.216507
      75%         0.315800
      max         0.619234
      Name: taux_hlm, dtype: float64
```

Le taux de logements sociaux (HLM) constitue un indicateur de la structure résidentielle et sociale des communes.

Cette variable peut être liée à certains comportements électoraux et à des caractéristiques socio-économiques spécifiques.

```
[294]: df_complet["total_chomeurs"] = (
      df_complet["chomeurs_15_24_ans"] +
      df_complet["chomeurs_25_54_ans"] +
      df_complet["chomeurs_55_64_ans"]
    )

df_complet["taux_chomage_global"] = (
      df_complet["total_chomeurs"] /
      df_complet["actifs_15_ans_plus"]
    )
df_complet["taux_chomage_global"].describe()
```

```
[294]: count      1476.000000
      mean         0.117221
      std          0.036221
      min          0.063063
      25%          0.085654
      50%          0.108344
      75%          0.142206
      max          0.218522
      Name: taux_chomage_global, dtype: float64
```

Le taux de chômage global permet de mesurer la proportion de chômeurs parmi la population active des communes.

Cet indicateur est particulièrement important dans l'analyse électorale, car les conditions économiques et l'emploi influencent souvent les comportements politiques et les tendances de vote.

```
[295]: df_complet.shape
```

```
[295]: (1476, 95)
```

Justification métier des nouvelles variables Les nouvelles variables créées permettent d'enrichir le dataset avec des indicateurs plus représentatifs des réalités territoriales, sociales et économiques des communes.

- **participation_reelle** : mesure le niveau de mobilisation électorale dans chaque commune. Une forte participation peut traduire un engagement citoyen plus important.
- **entreprises_par_habitant** : mesure le dynamisme économique local en rapportant le nombre d'établissements à la population.
- **associations_par_habitant** : représente l'intensité de la vie associative et l'engagement social local.
- **taux_hlm** : reflète la structure résidentielle et sociale des communes.
- **taux_chomage_global** : mesure la situation de l'emploi, facteur socio-économique pouvant influencer les comportements électoraux.
- **taux_logements_vacants** : indique l'attractivité résidentielle et certaines fragilités territoriales.

Ces variables permettent de transformer des données brutes en indicateurs métiers plus interprétables pour les modèles de Machine Learning.

9 9. Les corrélations

```
[296]: cols_num = df_complet.select_dtypes(include=["int64", "float64"]).copy()

      cols_to_exclude = ["code_commune", "code_postal"]

      cols_num = cols_num.drop(
          columns=cols_to_exclude,
          errors="ignore"
      )
```

```

print("Nombre de variables numériques :", cols_num.shape[1])

corr_matrix = cols_num.corr()

plt.figure(figsize=(18, 12))

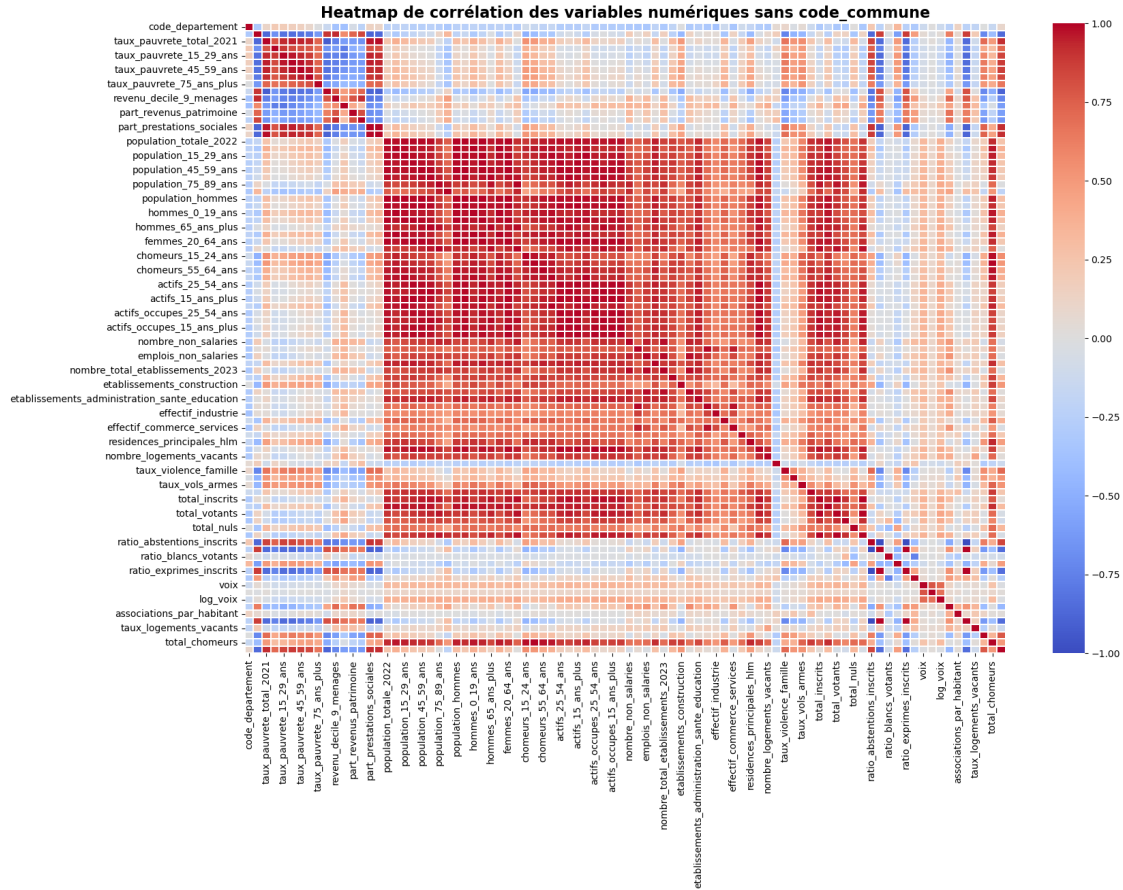
sns.heatmap(
    corr_matrix,
    cmap="coolwarm",
    center=0,
    linewidths=0.2
)

plt.title(
    "Heatmap de corrélation des variables numériques sans code_commune",
    fontsize=16,
    fontweight="bold"
)

plt.show()

```

Nombre de variables numériques : 88



```
[297]: corr_target = (
    df_complet
    .select_dtypes(include=["int64", "float64"])
    .corr()["log_voix"]
    .sort_values(ascending=False)
)

pd.set_option("display.max_rows", None)
print(corr_target)
```

```
log_voix          1.000000
voix              0.714093
pourcentage_voix  0.684084
total_votants     0.430093
total_exprimes    0.429891
total_inscrits    0.428503
femmes_65_ans_plus 0.418118
actifs_occupes_55_64_ans 0.416758
actifs_55_64_ans  0.411831
population_75_89_ans 0.411652
```

actifs_occupes_25_54_ans	0.411495
actifs_occupes_15_ans_plus	0.411272
nombre_salaries	0.410081
nombre_total_logements	0.406875
population_60_74_ans	0.404721
actifs_25_54_ans	0.404042
actifs_15_ans_plus	0.402869
hommes_65_ans_plus	0.401818
population_45_59_ans	0.398471
femmes_20_64_ans	0.397436
population_femmes	0.397209
total_blancs	0.391482
population_90_ans_plus	0.389462
population_30_44_ans	0.386335
population_totale_2022	0.385441
total_abstentions	0.384082
hommes_20_64_ans	0.382821
population_hommes	0.381565
nombre_non_salaries	0.378685
nombre_logements_vacants	0.377999
population_15_29_ans	0.370648
etablissements_administration_sante_education	0.368989
actifs_occupes_15_24_ans	0.365126
nombre_total_etablissements_2023	0.357441
femmes_0_19_ans	0.356949
population_0_14_ans	0.355316
hommes_0_19_ans	0.355297
nb_associations	0.350210
actifs_15_24_ans	0.348052
nombre_creations_entreprises_2024	0.346722
emplois_non_salaries	0.344621
chomeurs_55_64_ans	0.339361
etablissements_commerce_services	0.323585
total_chomeurs	0.305455
chomeurs_25_54_ans	0.303527
etablissements_industrie	0.303104
residences_principales_hlm	0.299864
emplois_salaries	0.287871
total_nuls	0.281127
chomeurs_15_24_ans	0.278359
effectif_total_salarie_2024	0.271926
effectif_commerce_services	0.244073
etablissements_construction	0.241290
effectif_administration_sante	0.238124
effectif_construction	0.232157
effectif_industrie	0.225922
part_revenus_activite	0.225230
taux_vols_armes	0.138187

entreprises_par_habitant	0.119353
revenu_decile_9_menages	0.107034
taux_violence_sexuelle	0.096926
taux_logements_vacants	0.090829
part_revenus_patrimoine	0.081106
ratio_exprimes_votants	0.063337
ratio_exprimes_inscrits	0.059924
ratio_votants_inscrits	0.058546
participation_reelle	0.058546
part_prestations_familiales	0.039029
taux_violence_famille	0.035206
revenu_median_menages_2021	0.033664
ratio_blancs_votants	0.023695
taux_hlm	0.007233
associations_par_habitant	-0.013647
revenu_decile_1_menages	-0.024127
taux_chomage_global	-0.027978
taux_pauvrete_total_2021	-0.052839
ratio_abstentions_inscrits	-0.058546
part_aides_logement	-0.063604
code_commune	-0.064905
part_prestations_sociales	-0.092568
taux_pauvrete_75_ans_plus	-0.100245
taux_pauvrete_60_74_ans	-0.102467
taux_cambriolage	-0.117950
ratio_nuls_votants	-0.118733
taux_pauvrete_45_59_ans	-0.119084
code_departement	-0.121018
code_postal	-0.122556
taux_pauvrete_30_44_ans	-0.133856
taux_pauvrete_0_14_ans	-0.142792
taux_pauvrete_15_29_ans	-0.156304

Name: log_voix, dtype: float64

```
[298]: corr_target = corr_matrix["log_voix"].sort_values(ascending=False)

print("Top 20 corrélations positives avec log_voix :")
print(corr_target.head(20))

print("\nTop 20 corrélations négatives avec log_voix :")
print(corr_target.tail(20))
```

```
Top 20 corrélations positives avec log_voix :
log_voix          1.000000
voix              0.714093
pourcentage_voix  0.684084
total_votants     0.430093
total_exprimes    0.429891
```

```

total_inscrits          0.428503
femmes_65_ans_plus      0.418118
actifs_occupes_55_64_ans 0.416758
actifs_55_64_ans        0.411831
population_75_89_ans    0.411652
actifs_occupes_25_54_ans 0.411495
actifs_occupes_15_ans_plus 0.411272
nombre_salaries         0.410081
nombre_total_logements  0.406875
population_60_74_ans    0.404721
actifs_25_54_ans        0.404042
actifs_15_ans_plus      0.402869
hommes_65_ans_plus      0.401818
population_45_59_ans    0.398471
femmes_20_64_ans        0.397436
Name: log_voix, dtype: float64

```

Top 20 corrélations négatives avec log_voix :

```

taux_violence_famille    0.035206
revenu_median_menages_2021 0.033664
ratio_blancs_votants     0.023695
taux_hlm                 0.007233
associations_par_habitant -0.013647
revenu_decile_1_menages  -0.024127
taux_chomage_global      -0.027978
taux_pauvrete_total_2021 -0.052839
ratio_abstentions_inscrits -0.058546
part_aides_logement      -0.063604
part_prestations_sociales -0.092568
taux_pauvrete_75_ans_plus -0.100245
taux_pauvrete_60_74_ans  -0.102467
taux_cambriolage         -0.117950
ratio_nuls_votants       -0.118733
taux_pauvrete_45_59_ans  -0.119084
code_departement         -0.121018
taux_pauvrete_30_44_ans  -0.133856
taux_pauvrete_0_14_ans   -0.142792
taux_pauvrete_15_29_ans  -0.156304
Name: log_voix, dtype: float64

```

```

[299]: top_positive = (
        corr_target
        .drop("log_voix")
        .sort_values(ascending=False)
        .head(20)
    )

```



```

top_negative = (
    corr_target
    .drop("log_voix")
    .sort_values(ascending=True)
    .head(20)
)

# Fusion des deux
top_corr_combined = pd.concat([
    top_positive,
    top_negative
])

top_corr_combined = top_corr_combined.sort_values()

```

```

[300]: plt.figure(figsize=(12,10))

colors = [
    "red" if val < 0 else "blue"
    for val in top_corr_combined
]

top_corr_combined.plot(
    kind="barh",
    color=colors
)

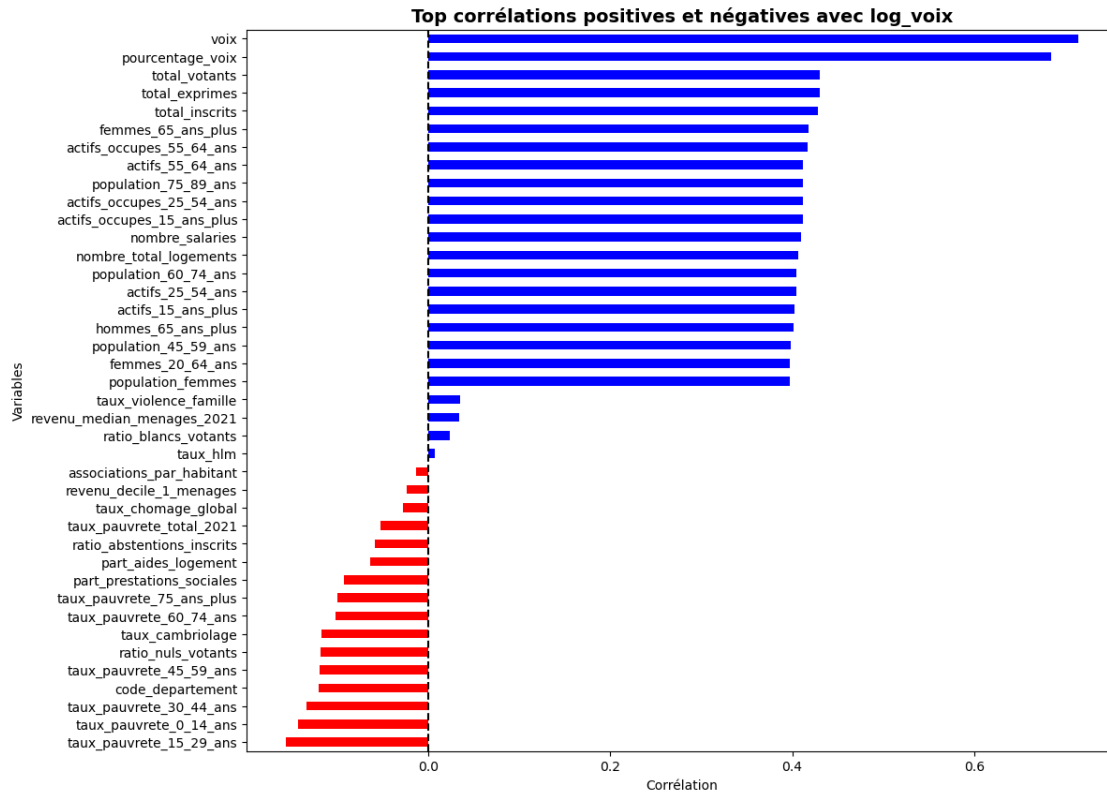
plt.title(
    "Top corrélations positives et négatives avec log_voix",
    fontsize=14,
    fontweight="bold"
)

plt.xlabel("Corrélation")
plt.ylabel("Variables")

plt.axvline(
    x=0,
    color="black",
    linestyle="--"
)

plt.show()

```



10. Préparation des données pour le Machine Learning

10.1 Définition des features et de la variable cible

```
[301]: selected_features = [

    # ===== DEMOGRAPHIE =====
    "population_totale_2022",
    "population_15_29_ans",
    "population_45_59_ans",
    "population_60_74_ans",

    # ===== ECONOMIE =====
    "revenu_median_ménages_2021",
    "nombre_total_etablissements_2023",
    "entreprises_par_habitant",

    # ===== EMPLOI =====
    "taux_chomage_global",

    # ===== SOCIAL =====
```

```

    "taux_pauvrete_total_2021",
    "taux_hlm",
    "associations_par_habitant",

    # ===== PARTICIPATION =====
    "participation_reelle",

    # ===== LOGEMENT =====
    "taux_logements_vacants",

    # ===== SECURITE =====
    "taux_cambriolage",

]

```

```

[302]: selected_corr = corr_target[selected_features]

selected_corr = selected_corr.sort_values()

plt.figure(figsize=(12,8))

colors = [
    "red" if val < 0 else "blue"
    for val in selected_corr
]

selected_corr.plot(
    kind="barh",
    color=colors
)

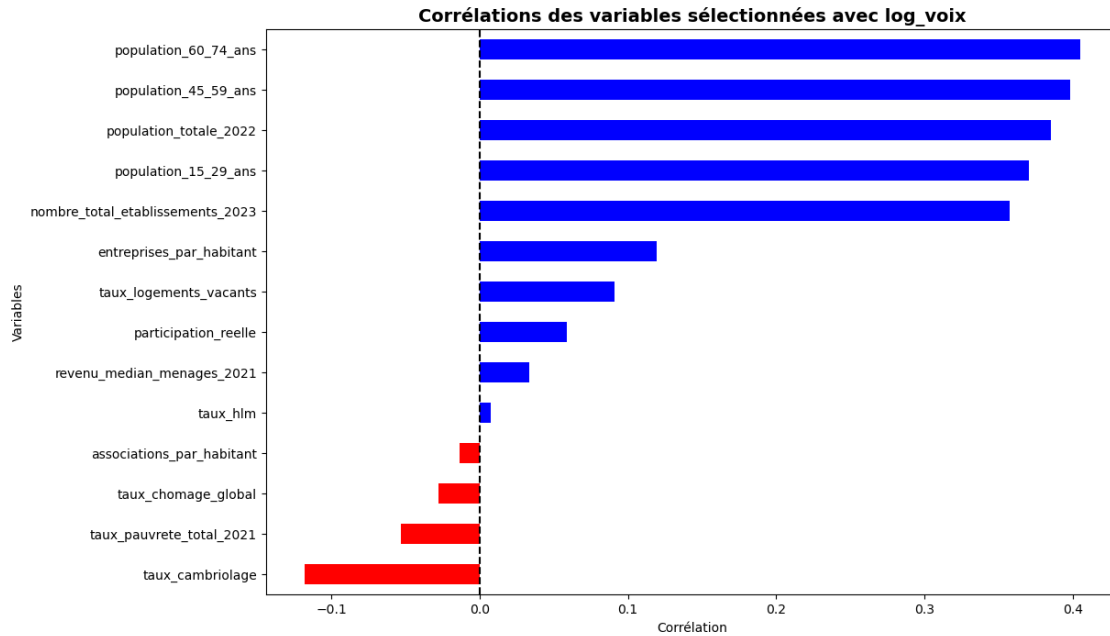
plt.title(
    "Corrélations des variables sélectionnées avec log_voix",
    fontsize=14,
    fontweight="bold"
)

plt.xlabel("Corrélation")
plt.ylabel("Variables")

plt.axvline(
    x=0,
    color="black",
    linestyle="--"
)

plt.show()

```



```
[303]: features = [
    # ===== CANDIDAT =====
    "nom_complet",

    # ===== DEMOGRAPHIE =====
    "population_totale_2022",
    "population_15_29_ans",
    "population_45_59_ans",
    "population_60_74_ans",

    # ===== ECONOMIE =====
    "revenu_median_menages_2021",
    "nombre_total_etablissements_2023",
    "entreprises_par_habitant",

    # ===== EMPLOI =====
    "taux_chomage_global",

    # ===== SOCIAL =====
    "taux_pauvrete_total_2021",
    "taux_hlm",
    "associations_par_habitant",

    # ===== PARTICIPATION =====
    "participation_reelle",
```

```
# ===== LOGEMENT =====
"taux_logements_vacants",

# ===== SECURITE =====
"taux_cambriolage",
]

target="log_voix"
```

10.2 Justification du choix des variables sélectionnées

La sélection des variables a été réalisée à partir de deux critères principaux :

- leur corrélation avec la variable cible `log_voix`,
- leur pertinence métier dans l'analyse des comportements électoraux.

L'objectif n'était pas uniquement de conserver les variables les plus corrélées statistiquement, mais également de construire un dataset équilibré représentant différentes dimensions socio-économiques des communes.

Il est également important de noter que la corrélation mesure principalement une relation linéaire simple entre deux variables. Ainsi, une faible corrélation n'implique pas nécessairement qu'une variable soit inutile pour le Machine Learning.

En effet, plusieurs modèles utilisés dans ce projet, notamment : - Random Forest, - XGBoost, - Decision Tree,

sont capables d'apprendre des relations complexes et non linéaires entre les variables explicatives et les comportements électoraux.

Certaines variables présentant une faible corrélation statistique avec `log_voix` peuvent donc rester très pertinentes d'un point de vue prédictif et métier.

10.2.1 Variables démographiques

Les variables liées à la population (`population_totale_2022`, `population_15_29_ans`, `population_45_59_ans`, `population_60_74_ans`) permettent de représenter la structure démographique des communes.

Les comportements électoraux pouvant varier selon les catégories d'âge, ces variables constituent des indicateurs importants pour les modèles prédictifs.

10.2.2 Variables économiques

Les variables économiques (`revenu_median_menages_2021`, `nombre_total_etablissements_2023`, `entreprises_par_habitant`) permettent d'évaluer : - le niveau de revenu, - le dynamisme économique, - l'activité entrepreneuriale des territoires.

Ces éléments peuvent influencer les préférences politiques et les résultats électoraux.

10.2.3 Variables sociales

Les variables sociales (`taux_pauvrete_total_2021`, `taux_hlm`, `associations_par_habitant`) permettent de représenter : - la fragilité sociale, - la structure résidentielle, - l'engagement citoyen local.

Même si certaines présentent une corrélation statistique faible avec `log_voix`, elles restent importantes d'un point de vue métier et politique.

10.2.4 Variables liées à l'emploi

Le `taux_chomage_global` constitue un indicateur socio-économique majeur pouvant influencer les comportements électoraux et les dynamiques politiques locales.

10.2.5 Variables électorales

La variable `participation_reelle` mesure le niveau de mobilisation électorale dans les communes et représente un facteur important dans l'analyse des résultats politiques.

10.2.6 Variables territoriales et de sécurité

Les variables `taux_logements_vacants` et `taux_cambriolage` permettent de représenter certaines réalités territoriales liées à l'attractivité résidentielle et au contexte sécuritaire local.

10.2.7 Variable candidat

La variable `nom_complet` a été conservée car l'approche du projet consiste à prédire le nombre de voix obtenu par chaque candidat dans chaque commune.

Les données socio-économiques étant identiques pour tous les candidats d'une même commune, le candidat constitue un facteur différenciant indispensable pour permettre au modèle de distinguer les différentes lignes du dataset.

Ainsi, la sélection finale des variables permet de construire un dataset cohérent, équilibré et adapté à la prédiction électorale par Machine Learning.

10.3 10.2 Séparation Train/Test par commune

```
[304]: communes = df_complet["libelle_commune"].unique()

train_communes, test_communes = train_test_split(
communes,
test_size=0.2,
random_state=42
)

train_df = df_complet[
df_complet["libelle_commune"].isin(train_communes)
].copy()

test_df = df_complet[
```

```

df_complet["libelle_commune"].isin(test_communes)
].copy()

print("Nombre de communes train :", train_df["libelle_commune"].nunique())
print("Nombre de communes test :", test_df["libelle_commune"].nunique())

print("\nNombre de lignes train :", train_df.shape)
print("Nombre de lignes test :", test_df.shape)

```

Nombre de communes train : 98
Nombre de communes test : 25

Nombre de lignes train : (1176, 95)
Nombre de lignes test : (300, 95)

10.4 10.3 Creation de X / y et encodage des variables

```

[305]: X_train = train_df[features].copy()
y_train = train_df[target].copy()

X_test = test_df[features].copy()
y_test = test_df[target].copy()

print("Avant encodage :")
print("X_train :", X_train.shape)
print("X_test :", X_test.shape)

X_train = pd.get_dummies(X_train)
X_test = pd.get_dummies(X_test)

X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

print("\nAprès encodage :")
print("X_train :", X_train.shape)
print("X_test :", X_test.shape)

print("\nTypes restants dans X_train :")
print(X_train.dtypes.value_counts())

```

Avant encodage :
X_train : (1176, 15)
X_test : (300, 15)

Après encodage :
X_train : (1176, 26)
X_test : (300, 26)

```
Types restants dans X_train :
float64      13
bool         12
int64         1
Name: count, dtype: int64
```

11 11 Modélisation Machine Learning

11.1 11.1 Définition des modèles

```
[306]: # Dictionnaire des modèles
models = {
    "Decision Tree": DecisionTreeRegressor(
        random_state=42
    ),

    "Random Forest": RandomForestRegressor(
        n_estimators=100,
        random_state=42
    ),

    "SVM": SVR(),

    "XGBoost": XGBRegressor(
        n_estimators=100,
        learning_rate=0.1,
        max_depth=4,
        random_state=42
    )
}
```

```
[307]: models
```

```
[307]: {'Decision Tree': DecisionTreeRegressor(random_state=42),
'Random Forest': RandomForestRegressor(random_state=42),
'SVM': SVR(),
'XGBoost': XGBRegressor(base_score=None, booster=None, callbacks=None,
        colsample_bylevel=None, colsample_bynode=None,
        colsample_bytree=None, device=None, early_stopping_rounds=None,
        enable_categorical=False, eval_metric=None, feature_types=None,
        feature_weights=None, gamma=None, grow_policy=None,
        importance_type=None, interaction_constraints=None,
        learning_rate=0.1, max_bin=None, max_cat_threshold=None,
        max_cat_to_onehot=None, max_delta_step=None, max_depth=4,
        max_leaves=None, min_child_weight=None, missing=nan,
        monotone_constraints=None, multi_strategy=None, n_estimators=100,
```



```
n_jobs=None, num_parallel_tree=None, ...))}
```

11.2 11.2 Entraînement et évaluation des modèles

```
[308]: resultats_models = []

for name, model in models.items():

    print("\n=====")
    print("Modèle :", name)
    print("=====")

    model.fit(X_train, y_train)

    test_temp = test_df.copy()
    test_temp["log_voix_predites"] = model.predict(X_test)

    pred_winners = test_temp.loc[
        test_temp.groupby("libelle_commune")["log_voix_predites"].idxmax()
    ].copy()

    real_winners_test = df_gagnant[
        df_gagnant["libelle_commune"].isin(test_communes)
    ].copy()

    df_compare = pred_winners.merge(
        real_winners_test,
        on="libelle_commune",
        how="inner",
        suffixes=("_pred", "_reel")
    )

    y_true = df_compare["famille_politique_reel"]
    y_pred = df_compare["famille_politique_pred"]

    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred, average="weighted",
    ↪ zero_division=0)
    recall = recall_score(y_true, y_pred, average="weighted", zero_division=0)
    f1 = f1_score(y_true, y_pred, average="weighted", zero_division=0)

    print("Accuracy :", accuracy)
    print("Precision :", precision)
    print("Recall :", recall)
    print("F1-score :", f1)

    print("\nClassification report :")
```

```

print(classification_report(y_true, y_pred, zero_division=0))

resultats_models.append({
    "Modele": name,
    "Accuracy": accuracy,
    "Precision": precision,
    "Recall": recall,
    "F1-score": f1
})

```

```

=====
Modèle : Decision Tree
=====
Accuracy : 0.88
Precision : 0.8835897435897436
Recall : 0.88
F1-score : 0.8803864734299516

```

```

Classification report :

```

	precision	recall	f1-score	support
Centre	0.83	0.91	0.87	11
Gauche	0.92	0.86	0.89	14
accuracy			0.88	25
macro avg	0.88	0.88	0.88	25
weighted avg	0.88	0.88	0.88	25

```

=====
Modèle : Random Forest
=====
Accuracy : 0.92
Precision : 0.92
Recall : 0.92
F1-score : 0.92

```

```

Classification report :

```

	precision	recall	f1-score	support
Centre	0.91	0.91	0.91	11
Gauche	0.93	0.93	0.93	14
accuracy			0.92	25
macro avg	0.92	0.92	0.92	25
weighted avg	0.92	0.92	0.92	25

=====

Modèle : SVM

=====

Accuracy : 0.56

Precision : 0.5646153846153846

Recall : 0.56

F1-score : 0.5614170692431562

Classification report :

	precision	recall	f1-score	support
Centre	0.50	0.55	0.52	11
Gauche	0.62	0.57	0.59	14
accuracy			0.56	25
macro avg	0.56	0.56	0.56	25
weighted avg	0.56	0.56	0.56	25

=====

Modèle : XGBoost

=====

Accuracy : 0.96

Precision : 0.9633333333333333

Recall : 0.96

F1-score : 0.9601288244766505

Classification report :

	precision	recall	f1-score	support
Centre	0.92	1.00	0.96	11
Gauche	1.00	0.93	0.96	14
accuracy			0.96	25
macro avg	0.96	0.96	0.96	25
weighted avg	0.96	0.96	0.96	25

[309]: *## 11.3 Comparaison des performances des modèles*

```
[310]: df_resultats_models = pd.DataFrame(resultats_models)

df_resultats_models = df_resultats_models.sort_values(
    by="Accuracy",
    ascending=False
)
```

```
print("\n==== COMPARAISON DES MODÈLES =====")
display(df_resultats_models)
```

```
==== COMPARAISON DES MODÈLES =====
```

	Modele	Accuracy	Precision	Recall	F1-score
3	XGBoost	0.96	0.963333	0.96	0.960129
1	Random Forest	0.92	0.920000	0.92	0.920000
0	Decision Tree	0.88	0.883590	0.88	0.880386
2	SVM	0.56	0.564615	0.56	0.561417

11.3 11.4 Sélection du meilleur modèle

```
[311]: df_resultats_models = df_resultats_models.sort_values(
        by="Accuracy",
        ascending=False
    ).reset_index(drop=True)

meilleur_modele_nom = df_resultats_models.loc[0, "Modele"]
meilleur_accuracy = df_resultats_models.loc[0, "Accuracy"]

print("Meilleur modèle :", meilleur_modele_nom)
print("Accuracy du meilleur modèle :", round(meilleur_accuracy, 4))

display(df_resultats_models)
```

Meilleur modèle : XGBoost

Accuracy du meilleur modèle : 0.96

	Modele	Accuracy	Precision	Recall	F1-score
0	XGBoost	0.96	0.963333	0.96	0.960129
1	Random Forest	0.92	0.920000	0.92	0.920000
2	Decision Tree	0.88	0.883590	0.88	0.880386
3	SVM	0.56	0.564615	0.56	0.561417

12 12. Visualisation des résultats Machine Learning

12.1 12.1 Comparaison des Accuracy des modèles

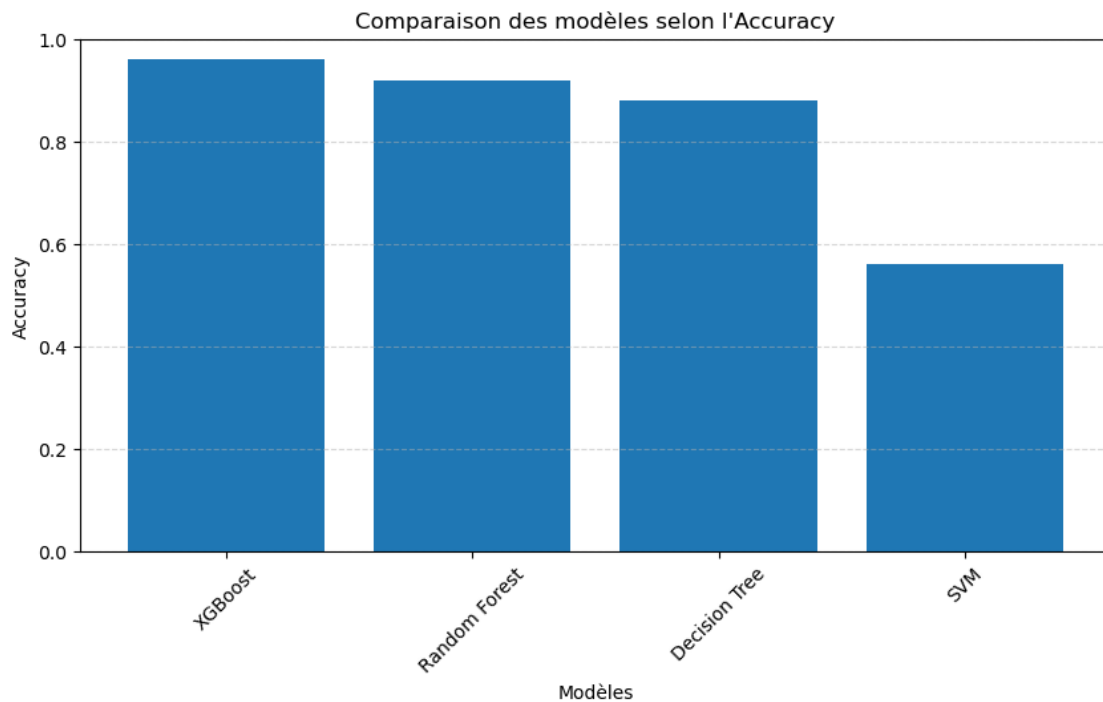
```
[312]: plt.figure(figsize=(10,5))

plt.bar(
    df_resultats_models["Modele"],
    df_resultats_models["Accuracy"]
)

plt.title("Comparaison des modèles selon l'Accuracy")
```

```
plt.xlabel("Modèles")
plt.ylabel("Accuracy")
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.grid(axis="y", linestyle="--", alpha=0.5)

plt.show()
```

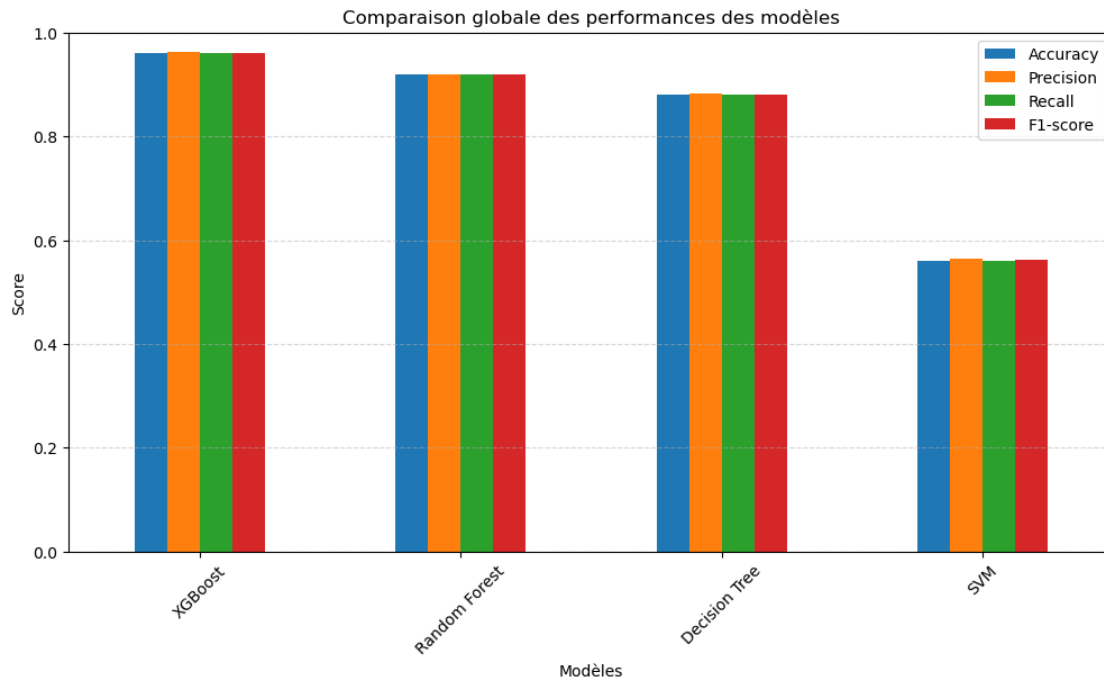


12.2 12.2 Comparaison globale des métriques

```
[313]: df_resultats_models.set_index("Modele") [
    ["Accuracy", "Precision", "Recall", "F1-score"]
].plot(
    kind="bar",
    figsize=(12,6)
)

plt.title("Comparaison globale des performances des modèles")
plt.xlabel("Modèles")
plt.ylabel("Score")
plt.xticks(rotation=45)
plt.ylim(0, 1)
plt.grid(axis="y", linestyle="--", alpha=0.5)
```

```
plt.show()
```



12.3 12.3 Matrice de confusion

```
[314]: best_model = models[meilleur_modele_nom]

best_model.fit(X_train, y_train)

test_temp = test_df.copy()
test_temp["log_voix_predites"] = best_model.predict(X_test)

pred_winners = test_temp.loc[
    test_temp.groupby("libelle_commune")["log_voix_predites"].idxmax()
].copy()

real_winners_test = df_gagnant[
    df_gagnant["libelle_commune"].isin(test_communes)
].copy()

df_compare_best = pred_winners.merge(
    real_winners_test,
    on="libelle_commune",
    how="inner",
    suffixes=("_pred", "_reel")
)
```

```

y_true_best = df_compare_best["famille_politique_reel"]
y_pred_best = df_compare_best["famille_politique_pred"]

cm = confusion_matrix(y_true_best, y_pred_best)

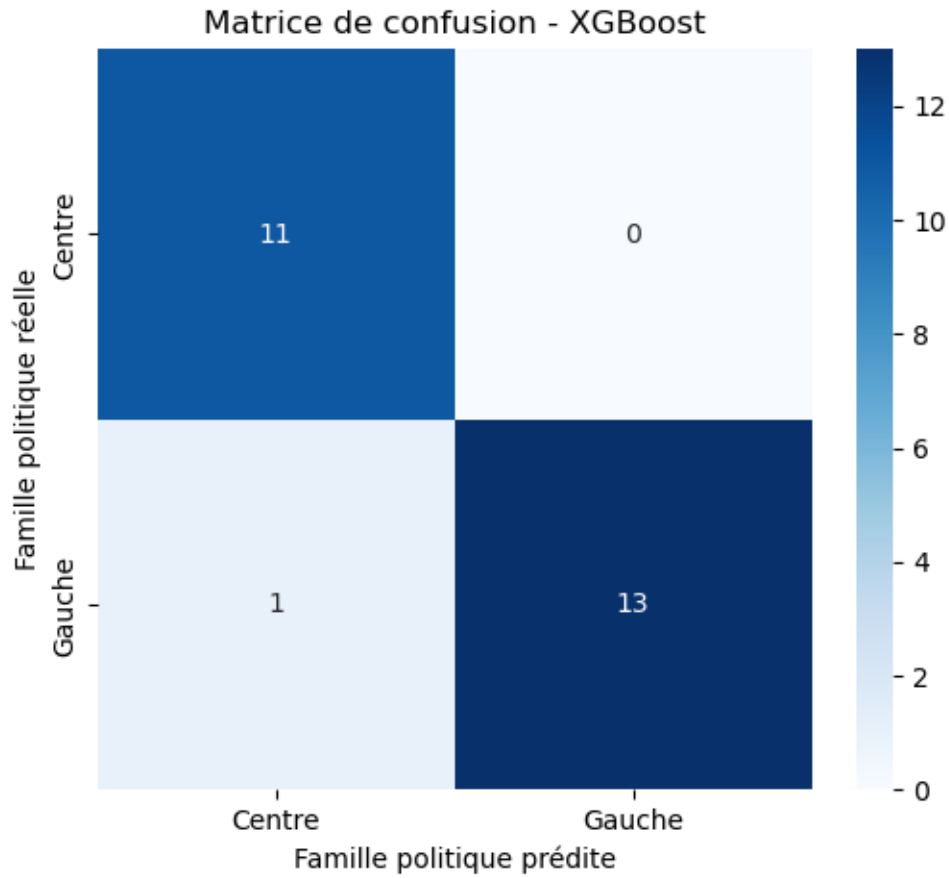
plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=sorted(y_true_best.unique()),
    yticklabels=sorted(y_true_best.unique())
)

plt.title(f"Matrice de confusion - {meilleur_modele_nom}")
plt.xlabel("Famille politique prédite")
plt.ylabel("Famille politique réelle")

plt.show()

```



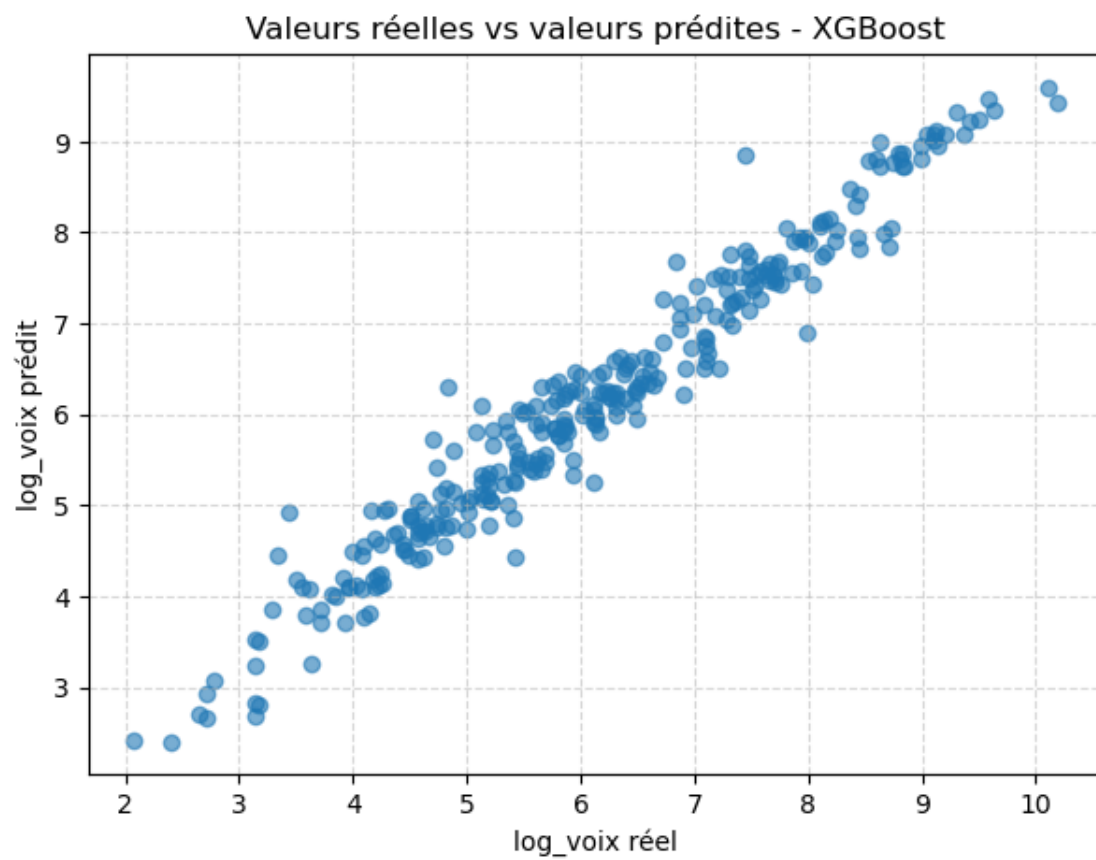
12.4 12.4 Scatter Plot : valeurs réelles vs prédictions

```
[315]: plt.figure(figsize=(7,5))

plt.scatter(
    y_test,
    best_model.predict(X_test),
    alpha=0.6
)

plt.title(f"Valeurs réelles vs valeurs prédites - {meilleur_modele_nom}")
plt.xlabel("log_voix réel")
plt.ylabel("log_voix prédit")
plt.grid(True, linestyle="--", alpha=0.5)

plt.show()
```

[]: